



UNIVERSITÀ DI PISA

Progettazione Sistemi Microelettronici
Modulo DIGITALE

Design and Implementation of an FPGA-based Programmable Pulse Sequencer

Professors: Luca Fanucci
Pietro Nannipieri

Student: Pietro Faraggiana

Contents

1	Introduction	1
2	Architectural Design	2
3	VHDL implementation	4
3.1	Random Acces Memory	4
3.2	Finite State Machine	5
3.2.1	Combinatorial Logic (Next-State Process)	6
3.2.2	Sequential Logic (State Update Process)	7
3.3	Timer	7
4	Verification and Simulation	8
4.1	Test Plan	8
4.2	Testbench 1: Base Sequence Without Feedback	10
4.3	Testbench 2: Error Conditions and Reset	11
4.4	Testbench 3: Feedback-Based Conditional Execution	13
5	Synthesis and Implementation Results	15
5.1	Synthesis and Implementation Flow	15
5.2	Resource Utilization	16
5.3	Timing and Critical Path Analysis	16
5.4	Power Consumption Analysis	17
5.5	Warnings	18
6	Conclusion	18
A	RAM	19
B	FSM	20
C	Timer	27
D	Top-Level PPS	28

1 Introduction

Quantum computing has been one of the most innovative fields in technology since Shor's algorithm showed its potential to solve problems intractable for classical computers[1].

A quantum computer works by manipulating multiple two-level quantum systems (qubits) that can be entangled with each other, enabling powerful parallel computations. The manipulation of qubits is carried out by precisely-timed pulses generated by specific hardware, which depends on the type of qubits used [2]. To execute a quantum algorithm, a compiler transforms the high-level code into a series of classical bits representing pulse instructions, which are then stored in a memory like a RAM. A quantum control unit (QCU) reads the instructions from the RAM sequentially, sending control signals to the waveform generators that produce the final pulses [3]. It is also possible to trigger

feedback mechanisms during the execution of the code; in this case, the QCU must also handle triggers and a feedback input and act accordingly.

To correctly interact with qubits, the generated pulses must be extremely precise. The timing of these pulses often requires a resolution on the order of picoseconds, and for real-time feedback operations, the total latency from measurement to the next control pulse must be on the order of tens to hundreds of nanoseconds [4].

The control pulses are essentially modulated high-frequency signals. A basic carrier wave can be described as a sinusoid:

$$A(t) \sin(2\pi f \cdot t + \phi) \quad (1)$$

Here, $A(t)$ is the amplitude, f is the frequency, and ϕ is the phase. In a programmable pulse sequencer (PPS), these three parameters are controlled digitally to shape the final pulse sent to the qubit. The amplitude determines the pulse's power, while frequency and phase are used to target specific qubits and perform different types of operations on their states. This level of control is fundamental for implementing quantum gates [5].

The following report details the first steps toward creating a PPS, a fundamental unit in quantum computers. Section 2 describes the architectural design, from high-level considerations to the detailed block diagrams of internal components. Section 3 details the VHDL implementation of each module. Section 4 presents the verification strategy, including the test plan and simulation results on ModelSim. Section 5 analyzes the results obtained from the synthesis and implementation on a Xilinx FPGA using Vivado, focusing on resource utilization, timing, and power. Finally, section 6 provides concluding remarks on the project, discusses its relevance in the context of state-of-the-art quantum control systems, and outlines potential future improvements.

2 Architectural Design

The PPS top-level block diagram is shown Figure 1.

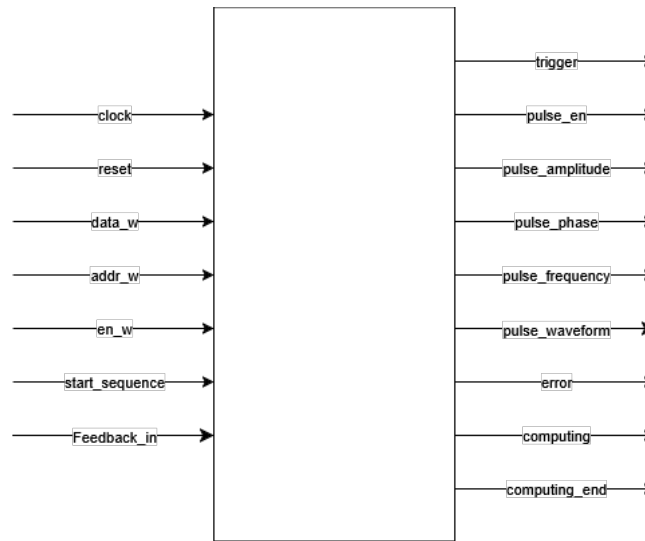


Figure 1: Top-Level block diagram of the PPS. The I/O are described in Table 1.

Table 1: Top-Level I/O Ports Description. The waveform parameter acts as the module dependency from time $A(t)$.

Port Name	Direction	Description
clk	Input	System clock signal.
reset	Input	Synchronous reset (active high).
start_sequence	Input	Starts the execution of the pulse sequence.
data_w	Input	Data bus for writing instructions to the RAM.
addr_w	Input	Address bus for writing instructions to the RAM.
en_w	Input	Write-enable for the RAM.
feedback_in	Input	Feedback input for conditional operations in the FSM.
pulse_en	Output	Pulse enable signal.
pulse_amplitude	Output	Digital value representing the pulse amplitude.
pulse_phase	Output	Digital value representing the pulse phase.
pulse_frequency	Output	Digital value representing the pulse frequency.
pulse_waveform	Output	Digital value indicating the waveform shape.
trigger	Output	Digital value indicating which trigger is active.
computing	Output	Flag indicating the sequencer is busy.
computing_end	Output	Flag indicating the end of the sequence has been reached.
error	Output	Flag indicating an error with the data written into the RAM.

The PPS was designed to be modular and scalable. As such, generics were heavily used and the top entity was divided into three main sub-modules as shown in Figure 2:

1. **Instruction Memory (RAM):** Stores the sequence of commands.
2. **Quantum Control Unit (FSM):** Reads, decodes, and executes the instructions.
3. **Timer:** Measures the duration for each command.

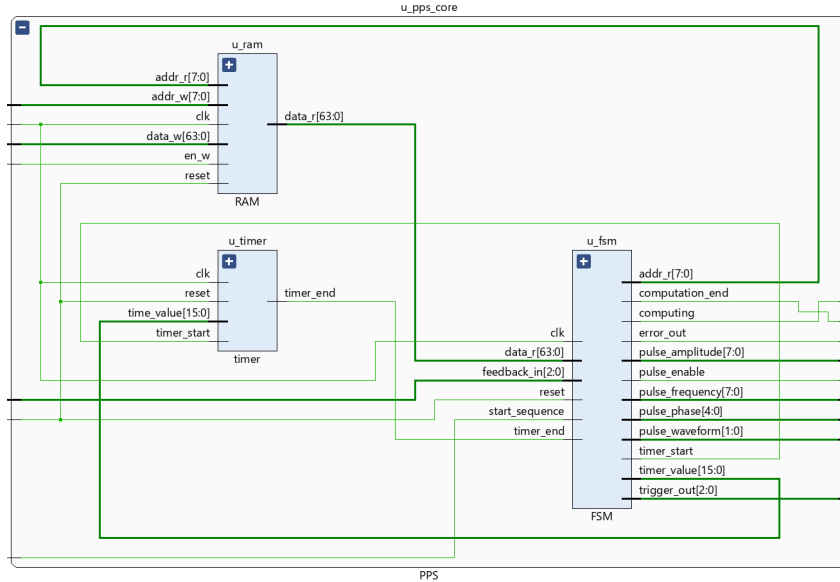


Figure 2: Internal architecture of the PPS.

The PPS operates by executing a pre-loaded program from its internal RAM. Even at the first start the PPS must receive a reset signal in order to work. Upon receiving a `start_sequence` signal, the FSM reads and decodes instructions sequentially. Each instruction dictates the parameters for pulse generation (amplitude, phase, frequency, waveform) or defines a delay period. The Timer module ensures that each operation lasts for the precise number of clock cycles specified in the instruction.

A key feature is the ability to handle conditional execution based on external feedback. This allows the sequencer to pause and wait for a measurement result before proceeding, a requirement for real-time algorithms like active qubit reset [3].

For the Quantum Control Unit (QCU), a custom Finite State Machine was chosen over a more general-purpose soft-processor like RISC-V. While RISC-V cores are common in advanced control systems for their flexibility [6], a dedicated FSM offers superior timing performance, which are the most critical requirements for this foundational pulse sequencing task. This choice also leads to a more lightweight and resource-efficient implementation on an FPGA. Furthermore, as this project focused exclusively on the programmable logic of the FPGA, a processor based architecture like RISC-V was outside the scope of the design.

The PPS outputs are digital control words intended for an Arbitrary Waveform Generator (AWG). Instead of generating raw waveform data, this system provides the high-level parameters that define the pulse envelope and carrier, a scalable approach common in quantum control hardware [2, 7].

3 VHDL implementation

3.1 Random Acces Memory

This block is a simple single-port RAM used to store the pulse sequence program. It allows writing new sequences from an external source and enables the FSM to read instructions during execution. The use of generics for data and address width makes the component highly configurable and scalable.

The core of the RAM is an array of `std_logic_vector`, followed by a signal to hold the data.

```

1 architecture behavioral of RAM is
2     type RAM_t is array (natural range 0 to 2**ADDR_WIDTH - 1) of
3         std_logic_vector(DATA_WIDTH - 1 downto 0);
4     signal ram_memory : RAM_t;
5 begin

```

Listing 1: RAM memory declaration

Both the write and read operations are implemented in a synchronous process that is sensitive to the clock's rising edge.

```

1 p_write : process (clk)
2 begin
3     if rising_edge(clk) then
4         if en_w = '1' then
5             ram_memory(to_integer(unsigned(addr_w))) <= data_w;
6         end if;
7     end if;
8 end process;

```

Listing 2: RAM write process

```

1 p_read : process (clk)
2 begin
3     if rising_edge(clk) then
4         if reset = '1' then
5             data_r <= (others => '0');

```

```

6         else
7             data_r <= ram_memory(to_integer(unsigned(addr_r)));
8         end if;
9     end if;
10 end process;
11 end architecture;

```

Listing 3: RAM read process

3.2 Finite State Machine

The Finite State Machine (FSM) is the core of the PPS. It acts as the central control unit, responsible for reading instructions from the RAM, decoding them, and orchestrating the operations of the other modules, such as the Timer and the pulse generation logic. The FSM's design is highly generic, allowing for easy adaptation to different instruction formats and pulse parameter widths.

The FSM implements a state machine that progresses through a defined sequence of states to execute a quantum program.

The FSM's behavior is defined by the states written in Listing 4 and separates the combinatorial and sequential logic in two different processes.

```

1  type state_t is (READY, READ_ADDR, READ_DATA, DECODE_INSTRUCTION,
2                  WAIT_FEEDBACK, PULSE, DELAY, WAIT_TIMER,
3                  DONE, ERROR);
4  signal current_state : state_t;
5  signal next_state   : state_t;

```

Listing 4: FSM states

- **READY**: The FSM has just been resetted and is ready to operate.
- **READ_ADDR**: The FSM is updating the `intruction_counter` register.
- **READ_DATA**: The FSM is updating the `intruction` register. These two state are needed to do this as the RAM needed a clock cycle to go to the next address.
- **DECODE_INSTRUCTION**: The FSM is decoding the received instruction.
- **WAIT_FEEDBACK**: If `feedback_mode = '1'` then we wait till the `feedback_in` $\neq$ `ZERO`.
- **PULSE**: If the decoded instruction was a pulse we set the right parameters.
- **DELAY**: If the decoded instruction was a delay we set the right parameters.
- **WAIT_TIMER**: The parameters previously set are mantained till the timer gets to '0'.
- **DONE**: The sequence has reached its end.
- **ERROR**: The instructions provided conteind an error of some kind.

Before any operation, the received instruction gets sliced into dedicated signals like opcode, duration, and amplitude.

```

1 decoded_opcode <= instruction(OPCODE_MSB downto OPCODE_LSB);
2 decoded_duration <= instruction(DURATION_MSB downto DURATION_LSB);
3 decoded_amplitude <= instruction(AMPLITUDE_MSB downto AMPLITUDE_LSB);
4 -- ... other field decodings

```

Listing 5: Concurrent instruction decoding

The combinatorial process manages the next state logic, while the sequential process manages the current state and registers updates.

3.2.1 Combinatorial Logic (Next-State Process)

The combinatorial process, `p_next_state`, calculates the next state and the outputs based on the current state and inputs.

The key state is `DECODE_INSTRUCTION`, where the FSM decides the next action based on the opcode. It branches to `WAIT_FEEDBACK` if the instruction requires conditional execution.

```

1 when DECODE_INSTRUCTION =>
2   if decoded_opcode = OP_END_SEQ then
3     next_state <= DONE;
4   elsif instruction_counter = ADDR_MAX then
5     next_state <= ERROR;
6   elsif decoded_feedback_mode = '1' then
7     next_state <= WAIT_FEEDBACK;
8   else
9     if decoded_opcode = OP_PULSE then
10      next_state <= PULSE;
11    elsif decoded_opcode = OP_DELAY then
12      next_state <= DELAY;
13    else
14      next_state <= ERROR;
15    end if;
16  end if;

```

Listing 6: Logic for the `DECODE_INSTRUCTION` state

The `WAIT_FEEDBACK` state is fundamental for implementing real-time conditional instructions. The FSM stalls in this state until a valid feedback signal is received. It compares the `memorized_feedback` (a register that memorizes the feedback input in case these goes back to 0 while it may still be required) with the expected `decoded_feedback_value`. If they match, or if the instruction has a "jolly" feedback value, the FSM proceeds to execute the pulse or delay. Otherwise, it increments the instruction counter and skips the current operation.

```

1 when WAIT_FEEDBACK =>
2   if feedback_received = '1' and
3     (memorized_feedback = decoded_feedback_value or
4     decoded_feedback_value = ZERO_FEEDBACK) then
5     if decoded_opcode = OP_PULSE then
6       next_state <= PULSE;
7     elsif decoded_opcode = OP_DELAY then
8       next_state <= DELAY;
9     end if;
10  elsif feedback_received = '1' then
11    next_state <= READ_ADDR;
12  else
13    next_state <= WAIT_FEEDBACK;

```

```
14 end if;
```

Listing 7: Logic for the WAIT_FEEDBACK state

3.2.2 Sequential Logic (State Update Process)

The sequential process, `p_state_update`, is triggered by the clock rising edge and is responsible for updating the FSM's state and internal registers.

```
1 if (current_state = WAIT_TIMER and timer_end = '1') or
2   (current_state = PULSE and decoded_duration = ZERO_DURATION) or
3   (current_state = DELAY and decoded_duration = ZERO_DURATION) or
4   (current_state = WAIT_FEEDBACK and next_state = READ_ADDR) then
5   if instruction_counter < ADDR_MAX then
6     instruction_counter <= instruction_counter + 1;
7   end if;
8 end if;
```

Listing 8: Instruction counter update logic

```
1 if (current_state = READY and next_state = READ_ADDR) then
2   memorized_feedback <= ZERO_FEEDBACK;
3   elsif current_state = DECODE_INSTRUCTION then
4     if decoded_feedback_mode = '0' then
5       memorized_feedback <= ZERO_FEEDBACK;
6     else
7       if feedback_in /= ZERO_FEEDBACK then
8         memorized_feedback <= feedback_in;
9       end if;
10    end if;
11 end if;
```

Listing 9: Memorized feedback update logic

```
1 if current_state = DECODE_INSTRUCTION and next_state = WAIT_FEEDBACK
   then
2   if feedback_in /= ZERO_FEEDBACK then
3     feedback_received <= '1';
4   else
5     feedback_received <= '0';
6   end if;
7 elsif current_state = WAIT_FEEDBACK then
8   if feedback_received = '0' and feedback_in /= ZERO_FEEDBACK then
9     feedback_received <= '1';
10    memorized_feedback <= feedback_in;
11  end if;
12  else
13    feedback_received <= '0';
14  end if;
15 end if;
```

Listing 10: feedback received update logic

3.3 Timer

The Timer module the last component of the PPS. It is completely controlled by the FSM, all input and output (except for the clock and the reset) are coming from or

going to the FSM. Its primary function is to generate precise time intervals, during the `WAIT_TIMER` state. The FSM provides a target duration value (`time_value`), a start input (`timer_start`) and the Timer counts down for the specified number of clock cycles. The timer is designed as a down counter, when the countdown completes, the Timer sends a `time_end` signal for a single clock cycle, effectively ending the `WAIT_TIMER` state.

The architecture uses two internal signals: `count` to hold the current countdown value and `counting` to act as an internal state flag, indicating whether the timer is active. This flag is essential to ensure the counter decrements only after being started and stops once it reaches zero.

```

1 if timer_start = '1' then
2     count <= unsigned(time_value);
3     if unsigned(time_value) > 0 then
4         counting <= '1';
5     else
6         counting <= '0';
7     end if;
8     elsif counting = '1' then
9         if count = 1 then
10            count <= (others => '0');
11            counting <= '0';
12        elsif count > 0 then
13            count <= count - 1;
14        end if;
15 end if;

```

Listing 11: Timer countdown process

The `timer_end` output signal is generated by a concurrent assignment. It is asserted for exactly one clock cycle under two conditions:

1. Instantly, if `timer_end = '0'`.
2. When `counting = '1'` `count = '1'`. This predictive logic ensures `timer_end` arrives to FSM without a one-cycle delay.

```

1 timer_end <= '1' when (timer_start = '1' and unsigned(time_value) = 0)
   or (counting = '1' and count = 1)
2     else '0';

```

Listing 12: `timer_end` logic

4 Verification and Simulation

After writing the VHDL code it is needed to verify that the circuit works accordingly. This was done by running simulations on Modelsim. This is an iterative process as simulations may reveal unwanted behaviours that require to change the source VHDL.

4.1 Test Plan

A small verification strategy was developed. The test plan is divided into three distinct testbenches, each designed to validate specific aspects of the PPS's functionality: from basic operation, to error cases, to feedback.

1. Testbench 1: Base Sequence Without Feedback

Objective: To verify the correct execution of a simple pulse sequence involving PULSE, DELAY, and END_SEQ instructions, without activating the feedback mode.

Checks:

- Verify that the FSM transitions through the states READY, READ_ADDR, READ_DATA, DECODE_INSTRUCTION, PULSE, WAIT_TIMER, and DELAY as expected.
- Verify that `instruction_counter` increases correctly.
- For PULSE and DELAY instructions, check that the output ports (`pulse_amplitude`, `pulse_phase`, etc.) match the values specified in the instruction word.
- Verify that the Timer works correctly.
- Verify that the correct behaviour upon reaching the OP_END_SEQ instruction.

2. Testbench 2: Error Conditions and Reset

Objective: To test the FSM's error handling capabilities and the functionality of the synchronous reset.

Checks:

- For the missing OP_END_SEQ scenario, verify that the FSM transitions to the ERROR state upon reaching the maximum address.
- When the `reset = '1'`, confirm that the FSM immediately returns to the READY state, the `instruction_counter` is cleared to zero, as all the pulse outputs.

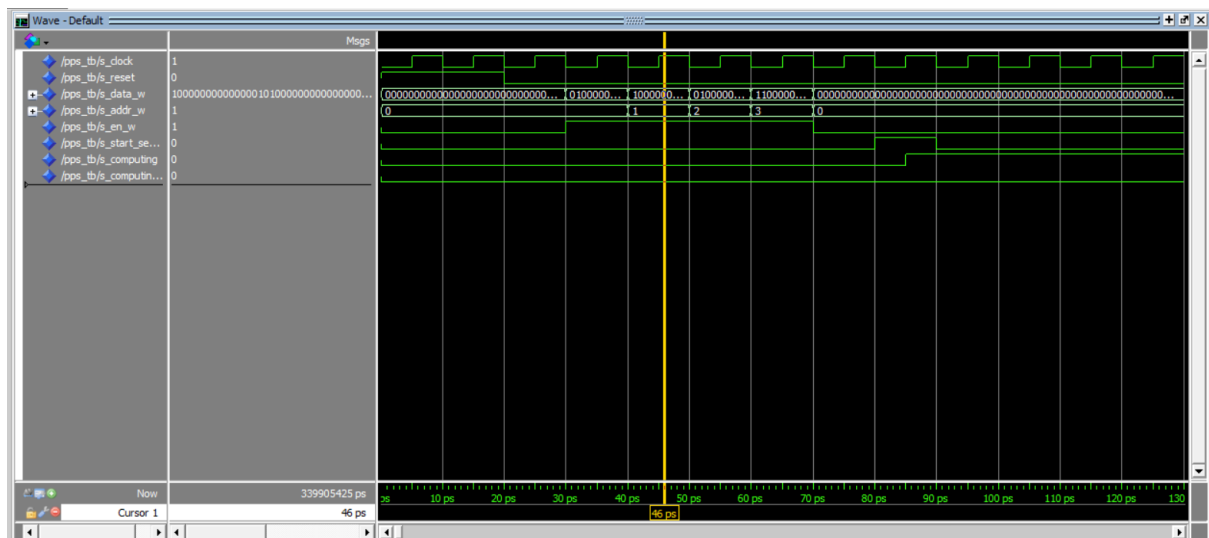
3. Testbench 3: Feedback-Based Conditional Execution

Objective: To thoroughly test the conditional execution logic controlled by the feedback mechanism.

Checks:

- Verify the correct entrance of the state WAIT_FEEDBACK upon receiving a feedback mode on instruction.
- Verify an instruction is executed when `memorized_feedback` matches `feedback_value`.
- Verify an instruction is executed when `feedback_value` is the "jolly".
- Verify that `memorized_feedback` does not follow `feedback_in = ZERO` when feedback mode is on.

The stimulus sequence is as follows:



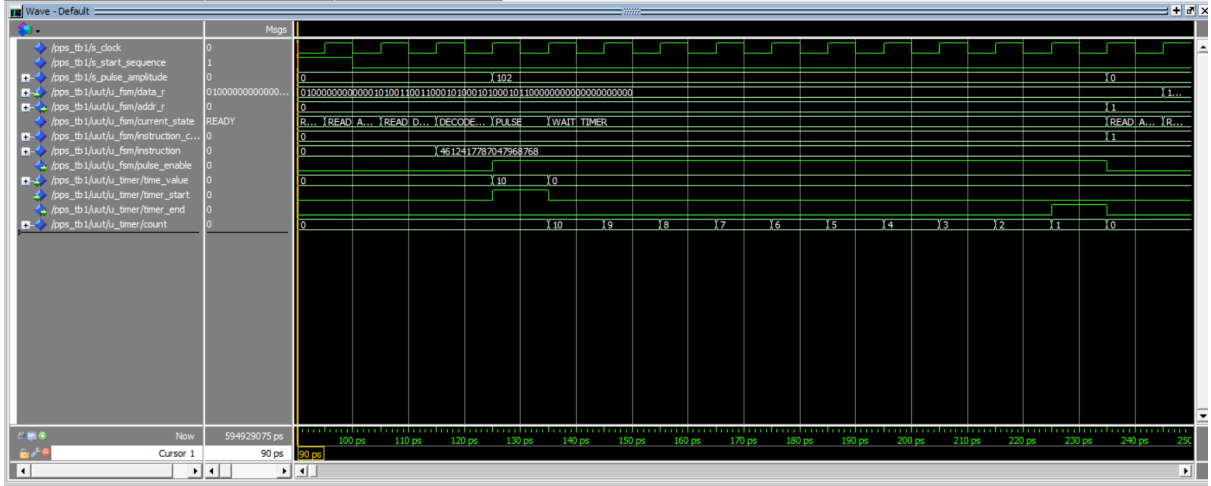


Figure 4: We can observe that the Timer lasts one cycle longer than the reported. This, although initially unwanted, was decided to be left on purpose because a pulse or delay of 0 duration is equivalent to skipping an instruction, which makes no sense. So it was decided to leave it, but it is important to be aware of it. The reason for this timing offset is that Pulse/Delay start at the same time as `timer_start = '1'`, but it takes one clock cycle for `timer_start` to start the timer. In order to eliminate this behavior it would be necessary to make the pulse wait one clock cycle before starting. In this snapshot we can verify the correct progression of the states.

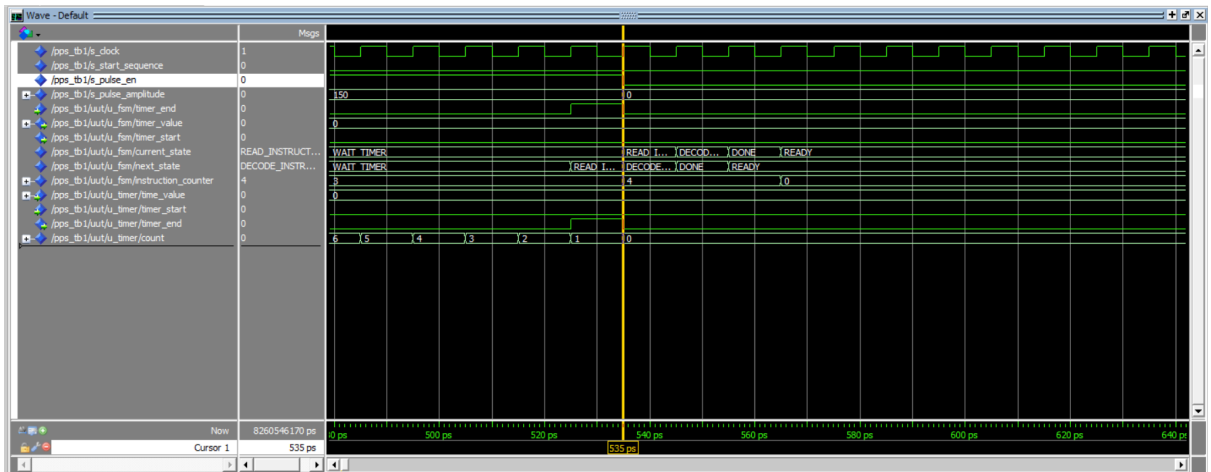


Figure 5: When the end sequence instruction is reached, the computation is over and the FSM is READY again.

4.3 Testbench 2: Error Conditions and Reset

The test is divided into two main scenarios:

1. No End-of-Sequence Error:

- After an initial reset, the testbench programs the entire RAM with a generic instruction. No `END_SEQ` instruction is written.
- The sequence is starts and the simulation runs long enough for the FSM to process all memory locations.

2. Reset Test:

- The PPS is resetted and the RAM is overwritten.
- While the PPS is in the middle of executing a long instruction, the PPS is resetted.

Watching the wave window in modelsim we can control the behaviour of each signal.

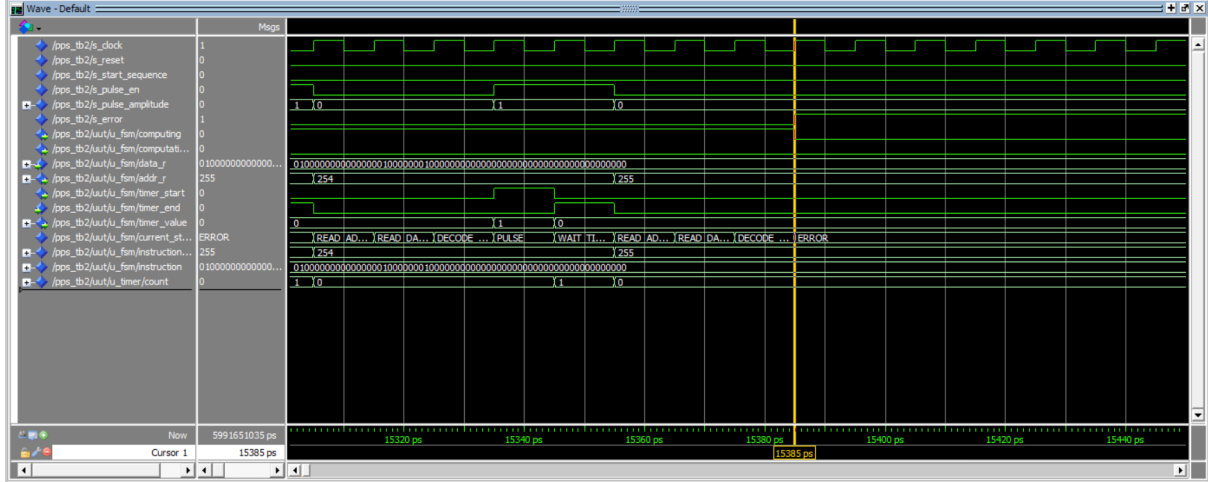


Figure 6: The last address of the RAM is reached and the last instruction is not a sequence end. The current state is ERROR. computing is off, but compute_end is not on, which is correct since the computation failed.

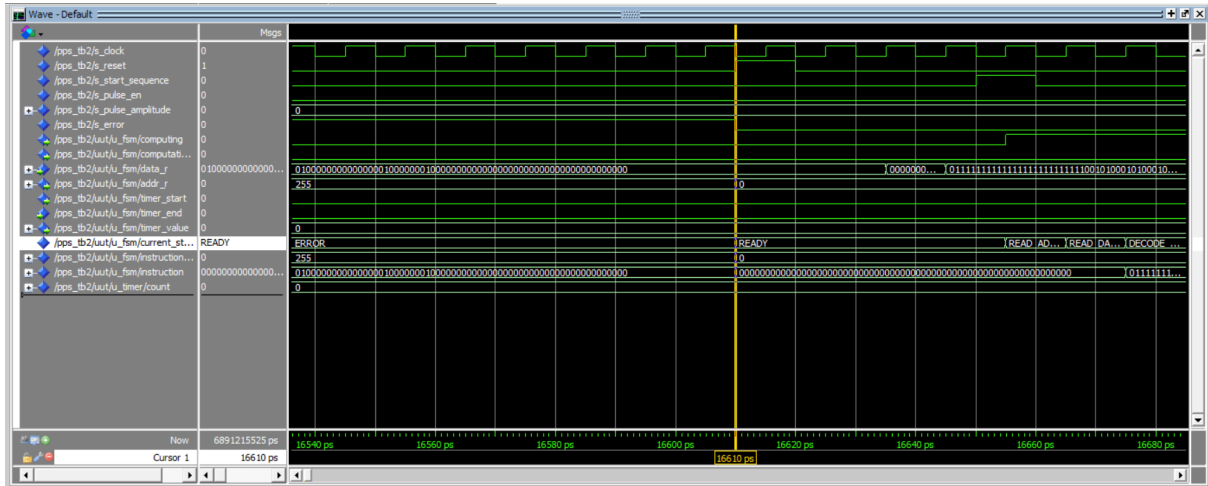


Figure 7: The PPS is resetted and the state is READY again. The RAM gets overwritten.

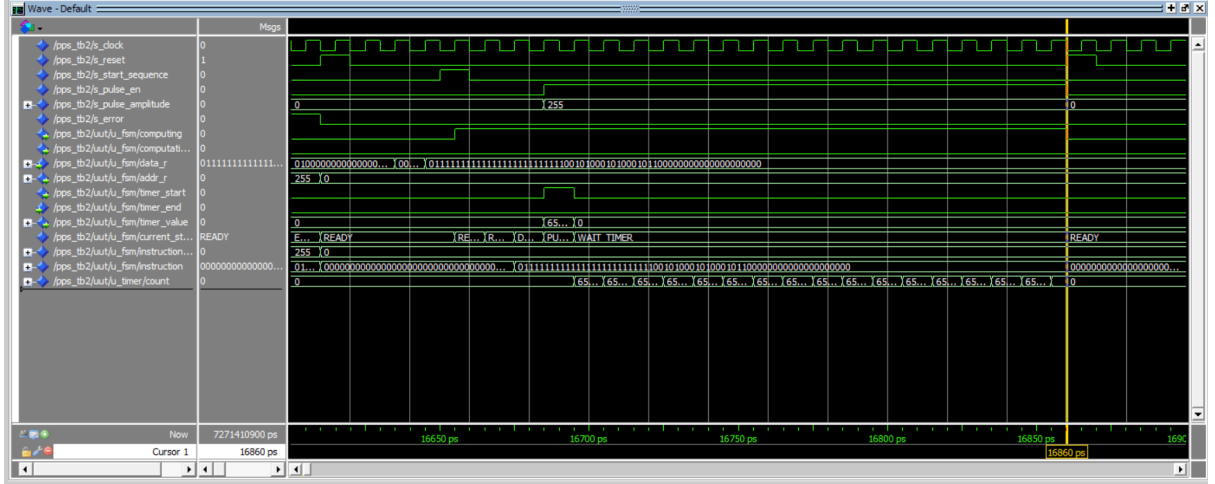


Figure 8: Reset during an instruction works well.

4.4 Testbench 3: Feedback-Based Conditional Execution

This final testbench is dedicated to verifying the most complex feature of the PPS: the conditional execution logic based on external feedback. To facilitate the synchronization of stimuli with the FSM’s internal state, a simple wrapper was created around the PPS design for this test. The wrapper exposes the FSM’s internal read address, allowing the testbench to apply the `feedback_in` signal at the precise moment the FSM is evaluating a specific conditional instruction.

The stimuli is as follows:

1. After a standard reset, the testbench loads some instructions into the RAM. This instructions includes a mix of conditional (`feedback_mode = '1'`) and non-conditional instructions, each crafted to test a specific aspect of the feedback logic.
2. When the FSM is about to process instruction 1 (which expects feedback "001"), the testbench provides the matching value "001" on `feedback_in`. This should cause the instruction to be executed normally.
3. Instruction 2 contains the "jolly" feedback value (all zeros). The testbench provides a non-zero feedback ("010"). This should cause the feedback
4. Instruction 3 expects feedback "010". The testbench first provides an incorrect value ("001"). This is expected to cause the FSM to skip instruction 3 and move to the next.
5. `feedback_in` is zeroed while an instruction has feedback mode on, `memorized_feedback` should memorize the last `feedback_in`.

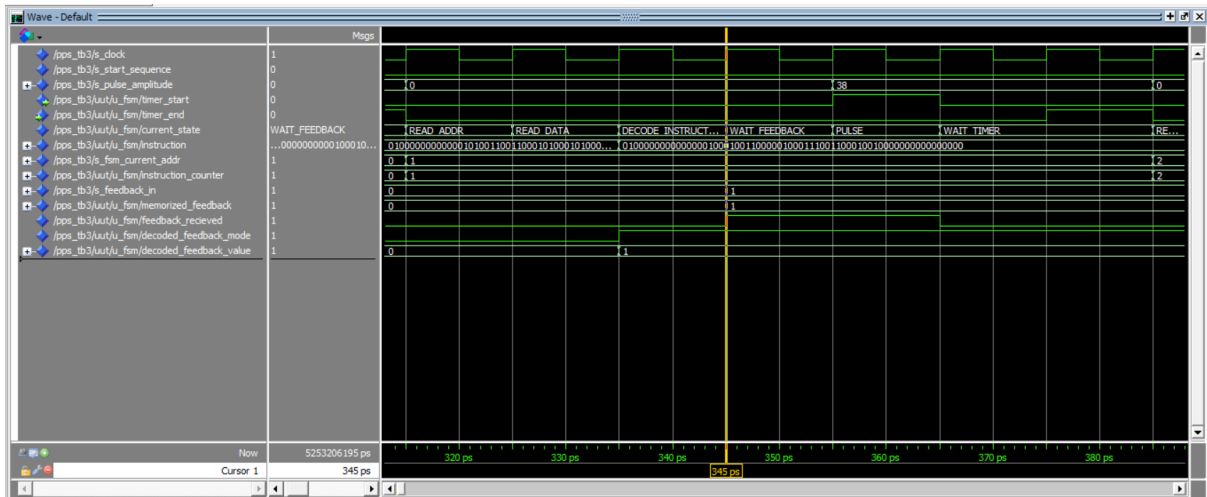


Figure 9: An instruction arrives with feedback mode on. The `feedback_value` bits and the `feedback_in` bits coincide so the instruction is executed.

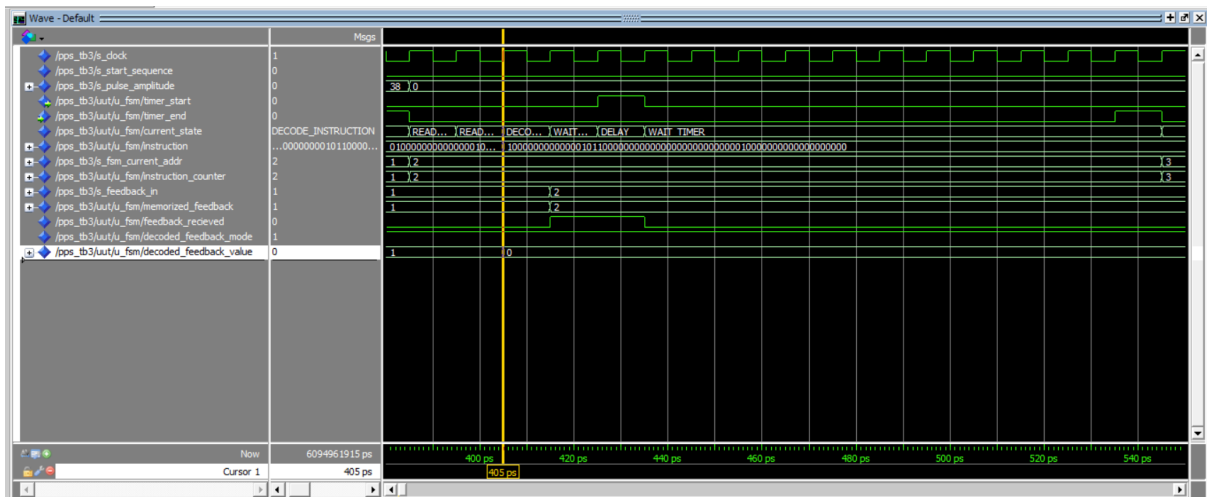


Figure 10: Feedback mode is still on, this time `feedback_value` and `feedback_in` do not coincide, but `feedback_value` is the "jolly" so the instruction is executed.

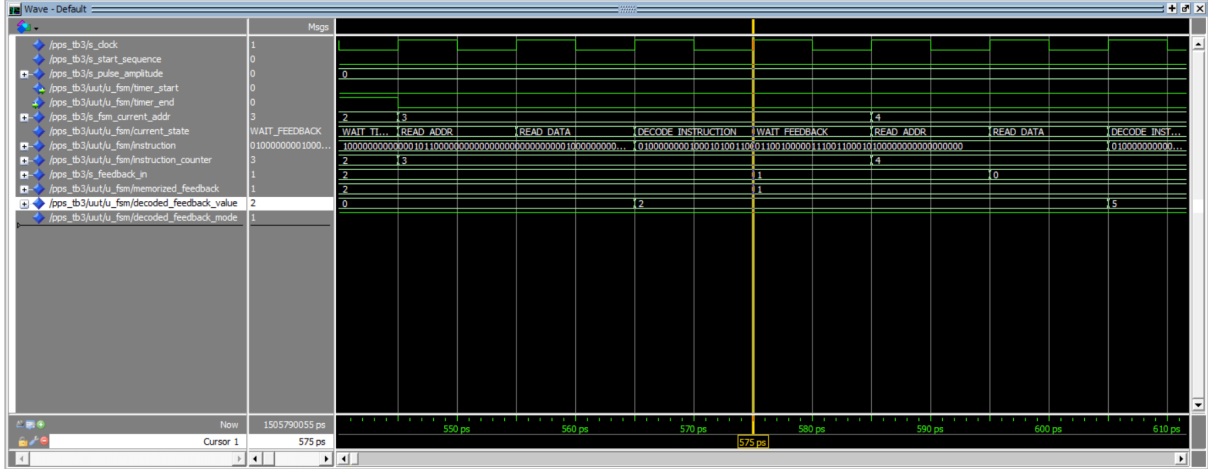


Figure 11: Again feedback mode is on, `feedback_value` and `feedback_in` do not coincide, but this time `feedback_value` = '000' so we skip the instruction and go to the next address. We can also observe that `memorized_feedback` that until now had followed the feedback input, but when the latter returns to 0 while feedback mode is on, then the data remains memorized.

5 Synthesis and Implementation Results

The synthesis and implementation of the PPS were performed using Vivado. This process is essential to analyze the circuit's real-world performance and to verify its optimization for a specific FPGA target. This software evaluates only the Register-Logic-Register paths, so it is necessary to "register" the inputs.

In order to do this a wrapper top-entity was created. The following processes ensures inputs receive datas only during the rising edge of the clock.

```

1 p_input_r : process (clk)
2 begin
3   if rising_edge(clk) then
4     if (reset = '1') then
5       else
6         s_data_w <= data_w;
7         s_addr_w <= addr_w;
8         -- ... other inputs
9       end if;
10    end if;
11 end process;

```

Listing 13: Input registers in the wrapper

A similar process is applied to the outputs.

5.1 Synthesis and Implementation Flow

PPS are typically implemented on high-performance RFSoc (Radio Frequency System-on-Chip) platforms, such as those from the Xilinx Zynq UltraScale+ family, which integrate both processing systems and advanced data converters on a single chip [7]. However, for this project, the available target devices within the Vivado Design Suite were limited to the Zynq-7000 family.

Within this family, the selection was narrowed down to devices with the highest speed grade available, specifically grade ‘-3’. Two main sub-families met this criteria:

- **Zynq-7020 (xc7z020...-3):**
 - LUTs: 53,200
 - Flip-Flops: 106,400
 - Block RAMs (36 Kb): 140
 - DSPs: 220
- **Zynq-7030 (xc7z030...-3):**
 - LUTs: 78,600
 - Flip-Flops: 157,200
 - Block RAMs (36 Kb): 265
 - DSPs: 400

The Zynq-7030 sub-family was ultimately chosen as the target for implementation. Although the resource utilization of the current PPS design is extremely little, selecting the device with greater available resources provides more flexibility for future expansions and the integration of more complex functionalities.

After choosing the FPGA Vivado elaborates the design, showing possible block implementations of the VHDL as shown in 1. It is possible to look inside these blocks to work out the elaborated design to a lower level, unfortunately the size of the design makes it hard to show them on a PDF.

The synthesis was run in an out of context mode, this means that the I/O pins were not requested to be declared. The only constraints requested was a period of 10ns.

5.2 Resource Utilization

The resource utilization report, generated post-implementation, confirms that the PPS design is extremely lightweight. As shown in Figure 12, the design consumes less than 1% of the available Look-Up Tables (LUTs), Flip-Flops (FFs), and Block RAMs (BRAMs) on the target Zynq-7030 device. This demonstrates the efficiency of the custom FSM-based architecture and indicates that there is ample room for future expansions.

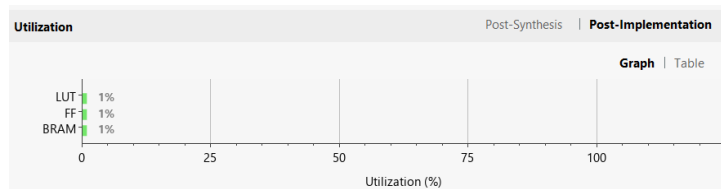


Figure 12: Post-implementation resource utilization.

5.3 Timing and Critical Path Analysis

The implementation was run with a target clock period of 10 ns (100 MHz). The post-implementation timing report is summarized in Figure 13.

Timing	
Worst Negative Slack (WNS):	6.304 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	191
Implemented Timing Report	

Figure 13: Post-Implementation Timing Summary.

The report shows a Worst Negative Slack (WNS) of 6.304 ns. The maximum achievable frequency (F_{max}) can be calculated from the WNS:

$$T_{min} = T_{required} - WNS = 3.696 \text{ ns} \quad (2)$$

$$F_{max} = \frac{1}{T_{min}} \approx 271 \text{ MHz} \quad (3)$$

The critical path (Figure 14) originates from the Block RAM's clock input and ends at a register within the FSM.

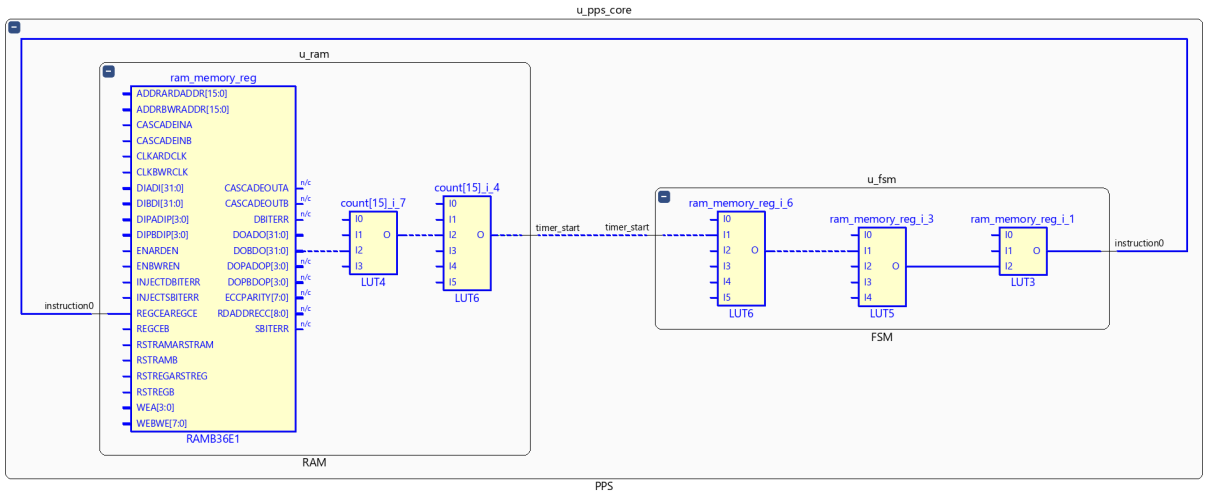


Figure 14: All the top 10 slower paths are RAM to FSM.

5.4 Power Consumption Analysis

An estimate of the power consumption is shown in Figure 15, the total on-chip power is estimated to be **0.128 W**.

Power	
Total On-Chip Power:	0.128 W
Junction Temperature:	25,2 °C
Thermal Margin:	74,8 °C (39,2 W)
Effective θ_{JA} :	1,9 °C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium
Implemented Power Report	

Figure 15: Post-implementation power consumption summary.

This low power figure is consistent with the minimal resource utilization and the moderate clock frequency.

5.5 Warnings

The implementation process was completed with a number of warnings.

DRC Violations	
Summary:	🚩 1 warning
Implemented DRC Report	

Figure 16: Design Rule Check (DRC) violations.

The DRC warning indicates that the Zynq Processing System block was not included in the design. This is expected and can be safely ignored as this project only implements the programmable logic of an FPGA.

81 implementation warnings arise, but they can all be safely ignored as they warn that clock and I/O pins are not connected (due to out of context mode) and this can lead to uncertainties on the timing analysis due to jitter and skew.

Implementation	
Status:	✅ Complete
Messages:	🚩 82 warnings
Part:	xc7z030fbg676-3
Strategy:	Vivado Implementation Defaults
Report Strategy:	Vivado Implementation Default Reports
Incremental implementation:	None

Figure 17: Implementation warnings.

6 Conclusion

The architecture was successfully implemented on a Xilinx Zynq-7000 series FPGA. The synthesis results demonstrate that the design is extremely resource-efficient, consuming

less than 1% of the target device’s logic and memory, and can operate reliably at frequencies up to 270 MHz. It’s important to understand that the data extrapolated from Vivado is highly dependent on the generics used.

As said in the introduction the current design is only a starting point for a working PPS. The timing precision of pulse generation should be enhanced to the picosecond level. This would necessitate more advanced platforms, such as an RFSoc. Furthermore, a detailed analysis of the critical timing paths, followed by the strategic insertion of pipeline stages, would be essential to increase the maximum operating frequency and reduce latency.

The architecture is inherently scalable. To control a system with N qubits, the design could be generalized by adding a few control bits to the instruction word to select the target qubit via a demultiplexer. For simultaneous control of multiple qubits, the instruction format could be expanded to include dedicated parameter fields for each of the N channels, allowing for parallel and independent pulse generation. This scalability, combined with the proven functionality of the core design, lays a solid foundation for developing more complex and capable quantum control systems.

References

- [1] Peter W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring".
- [2] Y. Xu, et al., "QubiC: An Open-Source FPGA-Based Control and Measurement System for Superconducting Quantum Information Processors"
- [3] N. Fruitwala, et al., "Distributed Architecture for FPGA-based Superconducting Qubit Control"
- [4] Y. Yang, et al., "FPGA-based electronic system for the control and readout of superconducting quantum processors"
- [5] Agilent Technologies, "The ABC's of Arbitrary Waveform Generation".
- [6] T. Guo, X. Guo, and M. Schulz, "An FPGA-based Quantum Control System with a Runtime Configurable Signal Generator"
- [7] U. Singhal, et al., "SQ-CARS: A Scalable Quantum Control and Readout System"

A RAM

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity RAM is
6     generic (
7         DATA_WIDTH : positive := 64; -- Larghezza della parola di dati
8         ADDR_WIDTH  : positive := 8  -- Larghezza dell'indirizzo
9     );
10    port (

```

```

11     clk    : in std_logic;
12     reset  : in std_logic;
13     -- Scrittura
14     en_w    : in std_logic;
15     addr_w  : in std_logic_vector(ADDR_WIDTH - 1 downto 0);
16     data_w  : in std_logic_vector(DATA_WIDTH - 1 downto 0);
17     -- Lettura
18     addr_r  : in std_logic_vector(ADDR_WIDTH - 1 downto 0);
19     data_r  : out std_logic_vector(DATA_WIDTH - 1 downto 0)
20     -- potrei aggiungere enable per la lettura per risparmiare potenza,
21     ma non lo faccio per evitare la complessit  aggiuntiva
22 );
23 end entity;
24
25 architecture behavioral of RAM is
26     -- creo le allocazioni per la memoria RAM
27     type RAM_t is array (natural range 0 to 2 ** ADDR_WIDTH - 1) of
28         std_logic_vector(DATA_WIDTH - 1 downto 0);
29     -- segnale per la memoria RAM
30     signal ram_memory : RAM_t;
31 begin
32     -- process per la scrittura
33     p_write : process (clk)
34     begin
35         if rising_edge(clk) then
36             if en_w = '1' then
37                 -- converto prima in unsigned e poi in integer per accedere all'
38                 array
39                 ram_memory(to_integer(unsigned(addr_w))) <= data_w;
40             end if;
41         end if;
42     end process;
43
44     -- process per la lettura
45     p_read : process (clk)
46     begin
47         if rising_edge(clk) then
48             if reset = '1' then
49                 data_r <= (others => '0');
50             else
51                 data_r <= ram_memory(to_integer(unsigned(addr_r)));
52             end if;
53         end if;
54     end process;
55 end architecture;

```

Listing 14: VHDL code for the RAM module.

B FSM

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity FSM is

```

```

6  generic (
7      ADDR_WIDTH           : positive := 8;
8      DATA_WIDTH          : positive := 64;
9      OPCODE_WIDTH         : positive := 2; -- larghezza dell'opcode
10     PULSE_DURATION_WIDTH  : positive := 16; -- risoluzione del timer
11     PULSE_AMPLITUDE_WIDTH : positive := 8; -- risoluzione dell'ampiezza
12     PULSE_PHASE_WIDTH     : positive := 5; -- risoluzione della fase
13     PULSE_FREQUENCY_WIDTH : positive := 8; -- risoluzione della
14     frequenza
15     PULSE_WAVEFORM_WIDTH  : positive := 2; -- numero potenza di due
16     forme d'onda
17     TRIGGER_WIDTH         : positive := 3; -- larghezza del segnale di
18     trigger
19     FEEDBACK_WIDTH        : positive := 3 -- larghezza del segnale di
20     feedback
21 );
22 port (
23     clk           : in std_logic;
24     reset         : in std_logic;
25     start_sequence : in std_logic;
26     error_out      : out std_logic;
27     computing      : out std_logic;
28     computation_end : out std_logic;
29     -- segnali per la RAM
30     data_r : in std_logic_vector(DATA_WIDTH - 1 downto 0);
31     addr_r : out std_logic_vector(ADDR_WIDTH - 1 downto 0);
32     -- segnali per il timer
33     timer_end : in std_logic;
34     timer_value : out std_logic_vector(PULSE_DURATION_WIDTH - 1 downto
35     0);
36     timer_start : out std_logic;
37     -- segnali per gli impulsi
38     pulse_amplitude : out std_logic_vector(PULSE_AMPLITUDE_WIDTH - 1
39     downto 0);
40     pulse_phase : out std_logic_vector(PULSE_PHASE_WIDTH - 1 downto
41     0);
42     pulse_frequency : out std_logic_vector(PULSE_FREQUENCY_WIDTH - 1
43     downto 0);
44     pulse_waveform : out std_logic_vector(PULSE_WAVEFORM_WIDTH - 1
45     downto 0);
46     pulse_enable : out std_logic;
47     -- segnali di trigger
48     trigger_out : out std_logic_vector(TRIGGER_WIDTH - 1 downto 0);
49     feedback_in : in std_logic_vector(FEEDBACK_WIDTH - 1 downto 0)
50 );
51 end entity;
52
53 architecture Behavioral of FSM is
54     -- stati della FSM
55     type state_t is (READY, READ_ADDR, READ_DATA, DECODE_INSTRUCTION,
56     WAIT_FEEDBACK, PULSE, DELAY, WAIT_TIMER, DONE, ERROR);
57     -- segnali interni
58     signal current_state : state_t; -- se non resettato all'inizio,
59     allora non entro mai in ready
60     signal next_state : state_t;
61     signal instruction_counter : unsigned(ADDR_WIDTH - 1 downto 0);
62     signal instruction : std_logic_vector(DATA_WIDTH - 1 downto
63     0);

```

```

52
53 signal decoded_opcode          : std_logic_vector(OPCODE_WIDTH - 1
54   downto 0);
54 signal decoded_duration        : std_logic_vector(PULSE_DURATION_WIDTH
55   - 1 downto 0);
55 signal decoded_amplitude       : std_logic_vector(
56   PULSE_AMPLITUDE_WIDTH - 1 downto 0);
56 signal decoded_phase           : std_logic_vector(PULSE_PHASE_WIDTH -
57   1 downto 0);
57 signal decoded_frequency       : std_logic_vector(
58   PULSE_FREQUENCY_WIDTH - 1 downto 0);
58 signal decoded_waveform        : std_logic_vector(PULSE_WAVEFORM_WIDTH
59   - 1 downto 0);
59 signal decoded_trigger         : std_logic_vector(TRIGGER_WIDTH - 1
60   downto 0);
60 signal decoded_feedback_mode    : std_logic;
61 signal decoded_feedback_value   : std_logic_vector(FEEDBACK_WIDTH - 1
62   downto 0);
62
63 -- segnali per la gestione del feedback
64 signal memorized_feedback : std_logic_vector(FEEDBACK_WIDTH - 1
65   downto 0);
65 signal feedback_received : std_logic;
66
67 -- costanti per gli Opcode
68 constant OP_PULSE : std_logic_vector(OPCODE_WIDTH - 1 downto 0) :=
69   "01";
69 constant OP_DELAY : std_logic_vector(OPCODE_WIDTH - 1 downto 0) :=
70   "10";
70 constant OP_END_SEQ : std_logic_vector(OPCODE_WIDTH - 1 downto 0) :=
71   "11";
71
72 -- | Opcode | Duration | Amplitude | Phase | Frequency | Waveform |
73   Trigger | Feedback | Unused |
73 constant OPCODE_MSB : natural := DATA_WIDTH - 1;
74 constant OPCODE_LSB : natural := OPCODE_MSB - OPCODE_WIDTH + 1;
75
76 constant DURATION_MSB : natural := OPCODE_LSB - 1;
77 constant DURATION_LSB : natural := DURATION_MSB -
78   PULSE_DURATION_WIDTH + 1;
78
79 constant AMPLITUDE_MSB : natural := DURATION_LSB - 1;
80 constant AMPLITUDE_LSB : natural := AMPLITUDE_MSB -
81   PULSE_AMPLITUDE_WIDTH + 1;
81
82 constant PHASE_MSB : natural := AMPLITUDE_LSB - 1;
83 constant PHASE_LSB : natural := PHASE_MSB - PULSE_PHASE_WIDTH + 1;
84
85 constant FREQUENCY_MSB : natural := PHASE_LSB - 1;
86 constant FREQUENCY_LSB : natural := FREQUENCY_MSB -
87   PULSE_FREQUENCY_WIDTH + 1;
87
88 constant WAVEFORM_MSB : natural := FREQUENCY_LSB - 1;
89 constant WAVEFORM_LSB : natural := WAVEFORM_MSB -
90   PULSE_WAVEFORM_WIDTH + 1;
90
91 constant TRIGGER_MSB : natural := WAVEFORM_LSB - 1;
92 constant TRIGGER_LSB : natural := TRIGGER_MSB - TRIGGER_WIDTH + 1;

```

```

93
94 constant FEEDBACK_MODE_MSB : natural := TRIGGER_LSB - 1;
95 constant FEEDBACK_MODE_LSB : natural := FEEDBACK_MODE_MSB;
96
97 constant FEEDBACK_VALUE_MSB : natural := FEEDBACK_MODE_LSB - 1;
98 constant FEEDBACK_VALUE_LSB : natural := FEEDBACK_VALUE_MSB -
    FEEDBACK_WIDTH + 1;
99
100 constant ADDR_MAX          : unsigned(ADDR_WIDTH - 1 downto 0)
    := to_unsigned(2 ** ADDR_WIDTH - 1, ADDR_WIDTH);
101 constant ZERO_FEEDBACK : std_logic_vector(FEEDBACK_WIDTH - 1 downto
    0) := (others => '0');
102 constant ZERO_DURATION : std_logic_vector(PULSE_DURATION_WIDTH - 1
    downto 0) := (others => '0');
103 begin
104
105 -- decodifica dell'istruzione
106 decoded_opcode      <= instruction(OPCODE_MSB downto OPCODE_LSB);
107 decoded_duration    <= instruction(DURATION_MSB downto
    DURATION_LSB);
108 decoded_amplitude   <= instruction(AMPLITUDE_MSB downto
    AMPLITUDE_LSB);
109 decoded_phase       <= instruction(PHASE_MSB downto PHASE_LSB);
110 decoded_frequency   <= instruction(FREQUENCY_MSB downto
    FREQUENCY_LSB);
111 decoded_waveform    <= instruction(WAVEFORM_MSB downto
    WAVEFORM_LSB);
112 decoded_trigger     <= instruction(TRIGGER_MSB downto TRIGGER_LSB)
    ;
113 decoded_feedback_mode <= instruction(FEEDBACK_MODE_MSB);
114 decoded_feedback_value <= instruction(FEEDBACK_VALUE_MSB downto
    FEEDBACK_VALUE_LSB);
115
116 -- process per i calcolo del prossimo stato
117 -- LOGICA COMBINATORIA
118 p_next_state : process (current_state, start_sequence, timer_end,
    data_r, instruction_counter,
119     decoded_opcode, decoded_duration, decoded_amplitude, decoded_phase,
120     decoded_frequency, decoded_waveform, decoded_trigger,
121     feedback_in, decoded_feedback_mode, decoded_feedback_value,
122     memorized_feedback, feedback_received)
123 begin
124     -- inizializzazione valori per evitare latch
125     next_state      <= current_state;
126     addr_r          <= std_logic_vector(instruction_counter);
127     timer_value     <= (others => '0');
128     timer_start     <= '0';
129     pulse_amplitude <= (others => '0');
130     pulse_phase     <= (others => '0');
131     pulse_frequency <= (others => '0');
132     pulse_waveform  <= (others => '0');
133     pulse_enable    <= '0';
134     trigger_out     <= (others => '0');
135     computing       <= '1';
136     computation_end <= '0';
137     error_out       <= '0';
138
139     case current_state is

```

```

140     when READY =>
141         computing <= '0';
142         if start_sequence = '1' then
143             next_state <= READ_ADDR;
144         end if;
145
146     when READ_ADDR =>
147         addr_r      <= std_logic_vector(instruction_counter);
148         next_state <= READ_DATA;
149
150     when READ_DATA => -- necessario per dare tempo a instruction di
aggiornarsi
151         next_state <= DECODE_INSTRUCTION;
152
153     when DECODE_INSTRUCTION =>
154         if decoded_opcode = OP_END_SEQ then
155             next_state <= DONE;
156         elsif instruction_counter = ADDR_MAX then -- se siamo qui
opcode non END_SEQ e se l'ultimo indirizzo, un errore.
157             next_state <= ERROR;
158         elsif decoded_feedback_mode = '1' then
159             if decoded_opcode = OP_PULSE or decoded_opcode = OP_DELAY
then
160                 next_state <= WAIT_FEEDBACK;
161             else
162                 next_state <= ERROR; -- Opcode non valido con feedback mode
163             end if;
164         else -- decoded_feedback_mode = '0'
165             if decoded_opcode = OP_PULSE then
166                 next_state <= PULSE;
167             elsif decoded_opcode = OP_DELAY then
168                 next_state <= DELAY;
169             else -- opcode non valido
170                 next_state <= ERROR;
171             end if;
172         end if;
173
174     when WAIT_FEEDBACK =>
175         if feedback_received = '1' and
176             (memorized_feedback = decoded_feedback_value or
decoded_feedback_value = ZERO_FEEDBACK) then
177             -- feedback ricevuto e corrisponde oppure jolly, procedi con
l'operazione
178             if decoded_opcode = OP_PULSE then
179                 next_state <= PULSE;
180             elsif decoded_opcode = OP_DELAY then
181                 next_state <= DELAY;
182             else
183                 next_state <= ERROR; -- Opcode non valido
184             end if;
185         elsif feedback_received = '1' then --feedback ricevuto ma
sbagliato
186             next_state <= READ_ADDR;
187         else
188             next_state <= WAIT_FEEDBACK;
189         end if;
190
191     when PULSE =>

```

```

192     pulse_amplitude <= decoded_amplitude;
193     pulse_phase     <= decoded_phase;
194     pulse_frequency <= decoded_frequency;
195     pulse_waveform  <= decoded_waveform;
196     trigger_out     <= decoded_trigger;
197     pulse_enable    <= '1';
198     if decoded_duration = ZERO_DURATION then
199         timer_start <= '0'; -- Non avviare il timer esterno
200         next_state  <= READ_ADDR; -- Passa subito alla prossima
istruzione (pulse di 0 cicli effettivi)
201     else
202         timer_value <= decoded_duration;
203         timer_start <= '1'; -- Avvia il timer esterno
204         next_state  <= WAIT_TIMER;
205     end if;
206
207 when DELAY =>
208     pulse_enable <= '0';
209     trigger_out  <= (others => '0');
210     if decoded_duration = ZERO_DURATION then
211         timer_start <= '0';
212         next_state  <= READ_ADDR;
213     else
214         timer_value <= decoded_duration;
215         timer_start <= '1';
216         next_state  <= WAIT_TIMER;
217     end if;
218
219 when WAIT_TIMER =>
220     if decoded_opcode = OP_PULSE then
221         pulse_enable <= '1';
222         pulse_amplitude <= decoded_amplitude;
223         pulse_phase <= decoded_phase;
224         pulse_frequency <= decoded_frequency;
225         pulse_waveform <= decoded_waveform;
226         trigger_out <= decoded_trigger;
227     else
228         pulse_enable <= '0';
229         trigger_out <= (others => '0');
230     end if;
231     timer_start <= '0';
232     if timer_end = '1' then
233         next_state <= READ_ADDR;
234     end if;
235 when DONE =>
236     computing <= '0';
237     computation_end <= '1';
238     next_state <= READY;
239
240 when ERROR =>
241     computing <= '0';
242     computation_end <= '0';
243     pulse_enable <= '0';
244     timer_start <= '0';
245     trigger_out <= (others => '0');
246     error_out <= '1';
247     next_state <= ERROR;
248 end case;

```

```

249 end process;
250
251 -- process per aggiornare lo stato
252 -- LOGICA SEQUENZIALE
253 p_state_update : process (clk)
254 begin
255     if rising_edge(clk) then
256         if reset = '1' then
257             current_state      <= READY;
258             instruction_counter <= (others => '0');
259             instruction         <= (others => '0');
260             memorized_feedback <= ZERO_FEEDBACK;
261             feedback_received  <= '0';
262         else
263
264             current_state <= next_state;
265
266             if current_state = READ_DATA and next_state =
267             DECODE_INSTRUCTION then
268                 instruction <= data_r;
269             end if;
270
271             if (current_state = READY and next_state = READ_ADDR) or (
272             current_state = DONE and next_state = READY) then -- inizio sequenza
273             , reset feedback memorizzato
274                 instruction_counter <= (others => '0');
275             elsif (current_state = WAIT_TIMER and timer_end = '1') or--
276             caumento indirizzo
277                 (current_state = PULSE and decoded_duration = ZERO_DURATION)
278             or
279                 (current_state = DELAY and decoded_duration = ZERO_DURATION)
280             or
281                 (current_state = WAIT_FEEDBACK and next_state = READ_ADDR)
282             then
283                 if instruction_counter < ADDR_MAX then
284                     instruction_counter <= instruction_counter + 1;
285                 end if;
286             end if;
287
288             -- Gestione di memorized_feedback
289             if (current_state = READY and next_state = READ_ADDR) then --
290             inizio sequenza, reset feedback memorizzato
291                 memorized_feedback <= ZERO_FEEDBACK;
292             elsif current_state = DECODE_INSTRUCTION then -- ogni volta che
293             siamo in decodifica
294                 if decoded_feedback_mode = '0' then --se feedback mode    0
295                 si resetta la memoria di feedback
296                     memorized_feedback <= ZERO_FEEDBACK;
297                 else-- decoded_feedback_mode = '1', memorizza il feedback
298                     if feedback_in /= ZERO_FEEDBACK then
299                         memorized_feedback <= feedback_in;
300                     end if;
301                 end if;
302             end if;
303
304             -- Gestione di feedback_received, necessario per rimanere in
305             WAIT_FEEDBACK
306             if current_state = DECODE_INSTRUCTION and next_state =

```

```

WAIT_FEEDBACK then
296     if feedback_in /= ZERO_FEEDBACK then
297         feedback_received <= '1';
298     else
299         feedback_received <= '0';
300     end if;
301     elsif current_state = WAIT_FEEDBACK then
302         if feedback_received = '0' and feedback_in /= ZERO_FEEDBACK
then
303             feedback_received <= '1';
304             memorized_feedback <= feedback_in; -- aggiungo cos posso
memorizzare anche in questo stato
305         end if;
306     else
307         feedback_received <= '0';
308     end if;
309 end if;
310 end if;
311 end process;
312 end architecture;

```

Listing 15: VHDL code for the FSM module.

C Timer

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity timer is
6     generic (
7         PULSE_DURATION_WIDTH : positive := 16
8     );
9     port (
10         clk          : in std_logic;
11         reset         : in std_logic;
12         time_value    : in std_logic_vector(PULSE_DURATION_WIDTH - 1 downto
0);
13         timer_start   : in std_logic;
14         timer_end     : out std_logic
15     );
16 end entity;
17
18 architecture Behavioral of timer is
19     -- segnali interni
20     signal count      : unsigned(PULSE_DURATION_WIDTH - 1 downto 0);
21     signal counting   : std_logic;
22
23 begin
24
25     p_timer : process (clk)
26     begin
27         if rising_edge(clk) then
28             if reset = '1' then
29                 count      <= (others => '0');
30                 counting <= '0';

```

```

31     else
32         if timer_start = '1' then
33             count <= unsigned(time_value);
34             if unsigned(time_value) = 0 then
35                 counting <= '0'; -- Non c'  bisogno di contare, evita
latch
36             else
37                 counting <= '1'; -- Inizia a contare
38             end if;
39             elsif counting = '1' then
40                 -- Se stiamo contando e il contatore 1, al prossimo colpo
di clock sar 0
41                 if count = 1 then
42                     count <= (others => '0');
43                     counting <= '0'; -- Smetti di contare, timer_end sar
attivo
44                 elsif count > 0 then -- basterebbe >1, ma non si sa mai
45                     count <= count - 1;
46                 else -- count 0, ma counting era 1. Resetta counting.
47                     counting <= '0';
48                 end if;
49             end if;
50             -- Se timer_start non  attivo e non stiamo contando, non fare
nulla
51             end if;
52         end if;
53     end process;
54
55     timer_end <= '1' when (timer_start = '1' and unsigned(time_value) =
0) or
56     (counting = '1' and count = 1)
57     else
58         '0';
59
60 end architecture;

```

Listing 16: VHDL code for the Timer module.

D Top-Level PPS

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity PPS is-- programmable pulse sequencer
6     generic (
7         ADDR_WIDTH           : positive := 8;
8         DATA_WIDTH          : positive := 64;
9         OPCODE_WIDTH         : positive := 2;
10        PULSE_DURATION_WIDTH  : positive := 16;
11        PULSE_AMPLITUDE_WIDTH : positive := 8;
12        PULSE_PHASE_WIDTH     : positive := 5;
13        PULSE_FREQUENCY_WIDTH : positive := 8;
14        PULSE_WAVEFORM_WIDTH  : positive := 2;
15        TRIGGER_WIDTH         : positive := 3;
16        FEEDBACK_WIDTH        : positive := 3

```

```

17 );
18 port (
19     clock          : in  std_logic;
20     reset          : in  std_logic;
21     data_w         : in  std_logic_vector(DATA_WIDTH - 1 downto 0)
22 ;
23     addr_w         : in  std_logic_vector(ADDR_WIDTH - 1 downto 0)
24 ;
25     en_w           : in  std_logic;
26     start_sequence : in  std_logic;
27     feedback_in    : in  std_logic_vector(FEEDBACK_WIDTH - 1
28     downto 0);
29     trigger        : out std_logic_vector(TRIGGER_WIDTH - 1 downto
30     0);
31     pulse_en       : out std_logic;
32     pulse_amplitude : out std_logic_vector(PULSE_AMPLITUDE_WIDTH -
33     1 downto 0);
34     pulse_phase    : out std_logic_vector(PULSE_PHASE_WIDTH - 1
35     downto 0);
36     pulse_frequency : out std_logic_vector(PULSE_FREQUENCY_WIDTH -
37     1 downto 0);
38     pulse_waveform  : out std_logic_vector(PULSE_WAVEFORM_WIDTH - 1
39     downto 0);
40     computing       : out std_logic;
41     computing_end   : out std_logic;
42     error           : out std_logic
43 );
44 end entity PPS;
45
46 architecture structural of PPS is
47
48     component FSM is
49         generic (
50             ADDR_WIDTH          : positive := 8;
51             DATA_WIDTH         : positive := 64;
52             OPCODE_WIDTH        : positive := 2;
53             PULSE_DURATION_WIDTH : positive := 16;
54             PULSE_AMPLITUDE_WIDTH : positive := 8;
55             PULSE_PHASE_WIDTH   : positive := 5;
56             PULSE_FREQUENCY_WIDTH : positive := 8;
57             PULSE_WAVEFORM_WIDTH : positive := 2;
58             TRIGGER_WIDTH       : positive := 3;
59             FEEDBACK_WIDTH      : positive := 3
60         );
61         port (
62             clk          : in  std_logic;
63             reset        : in  std_logic;
64             start_sequence : in  std_logic;
65             error_out     : out std_logic;
66             computing     : out std_logic;
67             computation_end : out std_logic;
68             data_r        : in  std_logic_vector(DATA_WIDTH - 1
69             downto 0);
70             addr_r        : out std_logic_vector(ADDR_WIDTH - 1
71             downto 0);
72             timer_end     : in  std_logic;
73             timer_value   : out std_logic_vector(PULSE_DURATION_WIDTH
74             - 1 downto 0);

```

```

64         timer_start      : out std_logic;
65         pulse_amplitude  : out std_logic_vector(
PULSE_AMPLITUDE_WIDTH - 1 downto 0);
66         pulse_phase      : out std_logic_vector(PULSE_PHASE_WIDTH -
1 downto 0);
67         pulse_frequency  : out std_logic_vector(
PULSE_FREQUENCY_WIDTH - 1 downto 0);
68         pulse_waveform   : out std_logic_vector(PULSE_WAVEFORM_WIDTH
- 1 downto 0);
69         pulse_enable     : out std_logic;
70         trigger_out      : out std_logic_vector(TRIGGER_WIDTH - 1
downto 0);
71         feedback_in     : in  std_logic_vector(FEEDBACK_WIDTH - 1
downto 0)
72     );
73     end component FSM;
74
75
76     component RAM is
77         generic (
78             DATA_WIDTH : positive := 64;
79             ADDR_WIDTH  : positive := 8
80         );
81         port (
82             clk      : in  std_logic;
83             reset    : in  std_logic;
84             en_w     : in  std_logic;
85             addr_w   : in  std_logic_vector(ADDR_WIDTH - 1 downto 0);
86             data_w   : in  std_logic_vector(DATA_WIDTH - 1 downto 0);
87             addr_r   : in  std_logic_vector(ADDR_WIDTH - 1 downto 0);
88             data_r   : out std_logic_vector(DATA_WIDTH - 1 downto 0)
89         );
90     end component RAM;
91
92     component timer is
93         generic (
94             PULSE_DURATION_WIDTH : positive := 16
95         );
96         port (
97             clk      : in  std_logic;
98             reset    : in  std_logic;
99             time_value : in  std_logic_vector(PULSE_DURATION_WIDTH - 1
downto 0);
100             timer_start : in  std_logic;
101             timer_end   : out std_logic
102         );
103     end component timer;
104
105
106     signal data_r      : std_logic_vector(DATA_WIDTH - 1 downto 0);
107     signal addr_r      : std_logic_vector(ADDR_WIDTH - 1 downto 0);
108     signal timer_end   : std_logic;
109     signal timer_value : std_logic_vector(PULSE_DURATION_WIDTH - 1
downto 0);
110     signal timer_start : std_logic;
111
112 begin
113     -- istanziazione della FSM

```

```

114 u_fsm : FSM
115     generic map (
116         ADDR_WIDTH          => ADDR_WIDTH ,
117         DATA_WIDTH         => DATA_WIDTH ,
118         OPCODE_WIDTH        => OPCODE_WIDTH ,
119         PULSE_DURATION_WIDTH => PULSE_DURATION_WIDTH ,
120         PULSE_AMPLITUDE_WIDTH => PULSE_AMPLITUDE_WIDTH ,
121         PULSE_PHASE_WIDTH   => PULSE_PHASE_WIDTH ,
122         PULSE_FREQUENCY_WIDTH => PULSE_FREQUENCY_WIDTH ,
123         PULSE_WAVEFORM_WIDTH => PULSE_WAVEFORM_WIDTH ,
124         TRIGGER_WIDTH        => TRIGGER_WIDTH ,
125         FEEDBACK_WIDTH       => FEEDBACK_WIDTH
126     )
127     port map (
128         clk          => clock ,
129         reset        => reset ,
130         start_sequence => start_sequence ,
131         error_out     => error ,
132         computing     => computing ,
133         computation_end => computing_end ,
134         data_r        => data_r ,
135         addr_r        => addr_r ,
136         timer_end     => timer_end ,
137         timer_value   => timer_value ,
138         timer_start   => timer_start ,
139         pulse_amplitude => pulse_amplitude ,
140         pulse_phase   => pulse_phase ,
141         pulse_frequency => pulse_frequency ,
142         pulse_waveform => pulse_waveform ,
143         pulse_enable  => pulse_en ,
144         trigger_out    => trigger ,
145         feedback_in    => feedback_in
146     );
147
148 -- Istanziamento della RAM
149 u_ram : RAM
150     generic map (
151         DATA_WIDTH => DATA_WIDTH ,
152         ADDR_WIDTH  => ADDR_WIDTH
153     )
154     port map (
155         clk    => clock ,
156         reset  => reset ,
157         en_w   => en_w ,
158         addr_w => addr_w ,
159         data_w => data_w ,
160         addr_r => addr_r ,
161         data_r => data_r
162     );
163
164 -- Istanziamento del Timer
165 u_timer : timer
166     generic map (
167         PULSE_DURATION_WIDTH => PULSE_DURATION_WIDTH
168     )
169     port map (
170         clk    => clock ,
171         reset  => reset ,

```

```
172         time_value => timer_value ,  
173         timer_start => timer_start ,  
174         timer_end   => timer_end  
175     );  
176 end architecture;
```

Listing 17: VHDL code for the top-level PPS module.